SOFTWARE TESTING (ISTQB CTFL) – GROUP PROJECT

LEARNING PROJECT REPORT

TOPIC 5

TEST TOOLS: REGRESSION TEST AUTOMATION

Comparing manual testing, capture/replay and data-driven automation on a product-search flow

Item	Information
Group number	_____
Course	Software Testing (ISTQB CTFL) – chapter 6, slides SWT6
Application under test	Shop web app (Next.js): https://my-app-nextjs-xi.vercel.app/ – source: github.com/pthaok6/LabFer202
Team repository	github.com/pthaok6/RBL_SWT
Lecturer	_____
Submission date	_____

Assignment of work

Member	Scope	Deliverables / evidence	Contribution
TV1 – _____	ISTQB theory; classification of 6 tool categories	01_LyThuyet.docx; 01_CongCu.xlsx	____ %
TV2 – _____	5 test cases; manual run; stopwatch timing	02_TestCases.xlsx; 02_ManualResults.csv	____ %
TV3 – _____	Capture/replay; run timing; locator-fix timing	03_CaptureReplay/ (scripts, logs)	____ %
TV4 – _____	Data-driven (CSV + pytest); run and locator-fix timing	04_DataDriven/ (scripts, CSV, logs)	____ %
TV5 – _____	Cost model; report; slides	05_BreakEven.xlsx; this report; slides	____ %

Names, student IDs and contribution percentages are left blank for the team to complete before submission.

Contents

1. Introduction
2. Theoretical background
3. Practice: design and implementation
4. Results and discussion
5. Conclusion and references
Appendix A. Test cases and evidence
Appendix B. Timing logs
Appendix C. How to reproduce

1. Introduction

Regression testing re-runs existing tests after a change to confirm that previously working functionality has not been broken. Because the same suite must be executed again and again, teams must decide whether to keep running it by hand or to invest in automation, which costs time to build and to maintain.

Objective. Automate a small regression suite in two ways – capture/replay and data-driven – and compare their cost with manual execution.

Main question. After how many runs does test automation become cheaper than manual execution?

System under test. A Shop web application built with Next.js and deployed at my-app-nextjs-xi.vercel.app (source: LabFer202). Only one flow is tested – product search – with five shared test cases, TC01–TC05. Figure 1 shows the search page.

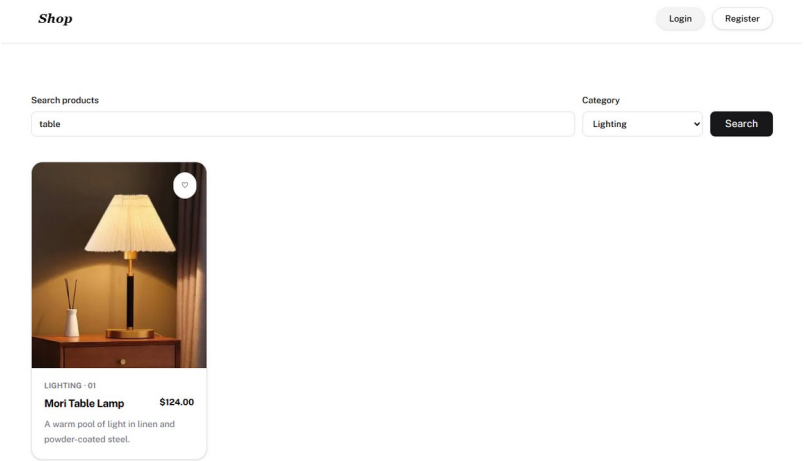


Figure 1. Search page of the application under test (evidence for TC03).

Scope of the work. The report covers the tool classification (task 1), the five test cases (task 2), the two automated suites (task 3), the locator-change experiment (task 4) and the break-even analysis with benefits and risks (task 5). One result is still missing: the manual run was not executed, so the manual time M has not been measured. This is stated wherever it matters, and no value has been invented to fill the gap.

2. Theoretical background

2.1 Regression testing and test automation

According to the ISTQB syllabus and the reference textbook (Black, van Veenendaal and Graham, Foundations of Software Testing, chapter 6, pp. 169–196), regression testing is the repeated execution of tests on a changed system to detect unintended side-effects. It differs from confirmation testing (re-testing), which only checks that a specific fixed defect is gone. Regression suites are the typical candidate for automation: the steps are stable, repeated often and benefit from consistent execution. Automation is not free, however: scripts must be built (initial effort) and updated when the application changes (maintenance).

2.2 Classification of test tools

ISTQB groups tools by the test activity they support. Table 1 lists one representative tool for each of the six categories required by the assignment (details, licences and references are in 01_CongCu.xlsx).

ISTQB category	Tool	Purpose
Test management and testware	Jira + Xray (or TestRail)	Manage test cases, runs and defects; requirements traceability and reporting.
Static testing	SonarQube	Analyse source code without executing it; find bugs, vulnerabilities, code smells.
Test design and implementation	GraphWalker	Model-based testing: generate test paths from a state-machine model.
Test execution and logging	Selenium WebDriver	Drive the browser, simulate user actions and compare actual with expected results.
Performance and monitoring	Apache JMeter	Generate load with virtual users; measure response time and throughput.
Specialised testing (security)	OWASP ZAP	Scan web applications for security vulnerabilities (passive and active scan).

Table 1. Six tool categories with one representative tool each.

The tools used in this project, Playwright and pytest, belong to the test execution category.

2.3 Capture/replay and data-driven automation

Capture/replay records the tester's actions in the browser (click, type, select) and generates a linear script that can be replayed. It is quick to start but, as the course slides point out, it does not scale, keeps test data hard-coded inside the script and breaks easily when the interface changes. **Data-driven testing** separates a generic control script from the test data (inputs and expected results) stored in an external file such as CSV; the script reads each row and runs the same logic. New cases are added as rows, and locators can be centralised in one place.

Criterion	Capture/replay	Data-driven
Initial effort (S)	Low – record and refine	Higher – design data reader, assertions, page object
Skills needed	Basic	Programming and test design
Data and logic	Data hard-coded in the script	Data in CSV, one generic script

Criterion	Capture/replay	Data-driven
Robustness to UI change	Fragile; edits spread over scripts	Locators in one file
Maintenance (R)	Higher	Lower

Table 2. Qualitative comparison of the two approaches (theory; measured values in Section 4).

2.4 Benefits and risks of tools (SWT6, slides 37–39)

Potential benefits: less repetitive work; greater consistency and repeatability; objective assessment (pass/fail and timing are data); easy access to test information and reports.

Risks: unrealistic expectations of the tool; underestimating the time, cost and effort of introducing it and of maintaining the scripts – exactly the parameters S and R of our model; over-reliance on the tool at the expense of exploratory testing; difficulties of integration and interoperability; vendor or open-source project risks.

2.5 Cost model and break-even point

Let **M** be the manual time per run, **A** the automated time per run, **S** the one-off build time and **R** the maintenance time amortised per run. The cumulative costs after N runs are C_manual(N) = N × M and C_auto(N) = S + N × (A + R). Setting them equal gives the break-even point

N_BE = S / (M − A − R), valid only when M > A + R.

If M ≤ A + R, automation never pays back in time. The smallest whole number of runs for which automation is strictly cheaper is floor(N_BE) + 1. R is computed as R = R_locator / K, where R_locator is the measured time to repair the locators after a UI change and K is the assumed number of runs between two UI changes; the locator-fix time is counted only inside R and never added a second time.

2.6 The seven testing principles

ISTQB summarises general testing knowledge in seven principles. Table 1b lists them; Section 4.3 links principles 1, 5 and 6 to the measured results.

No.	Principle	Short meaning
1	Testing shows the presence of defects	Passing tests does not prove the absence of defects.
2	Exhaustive testing is impossible	Choose tests by risk and priority, not by trying every input.
3	Early testing saves time and money	Start testing activities as early as possible.
4	Defects cluster together	A few modules usually contain most defects.
5	Beware of the pesticide paradox	Repeating the same tests stops finding new defects; tests must be reviewed and refreshed.
6	Testing is context dependent	The approach depends on the type of system and project.
7	Absence-of-errors fallacy	A system with few defects can still fail users if it does not meet their needs.

Table 1b. The seven ISTQB testing principles.

3. Practice: design and implementation

3.1 Environment and controlled variables

Parameter	Value actually used
Application / URL	Shop (LabFer202), https://my-app-nextjs-xi.vercel.app/
Flow and test cases	Product search; TC01–TC05, identical for all methods
Browser	Chromium, headless, driven by Playwright
Capture/replay runner	Playwright Test (JavaScript, Node.js) – search_products.spec.js
Data-driven runner	pytest 9.2.0 + Playwright for Python 1.64.1, Python 3.12.10, Windows
Measurement dates	Capture/replay benchmark: 2026-11-03; data-driven benchmark: 2026-11-07
Repetitions and unit	3 consecutive runs of the whole 5-case suite; seconds (build and repair times in minutes)
Timing rule	Execution time of each test case as logged by the runner_benchmark scripts; setup and repair times recorded separately

Table 3. Test environment and controlled variables.

3.2 Shared test suite

ID	Name	Input (keyword / category)	Expected result
TC01	Exact Name Search	Ripple Glass Set / All	Product “Ripple Glass Set” is displayed
TC02	Partial Name Search	lamp / All	“Mori Table Lamp” appears (case-insensitive)
TC03	Keyword and Category Filter	table / Lighting	“Mori Table Lamp” appears
TC04	Non-Existent Product	Laptop Dell XPS / All	Message “No products found.”
TC05	Empty Keyword	(empty) / All	All 6 products are displayed

Table 4. The five test cases shared by all three methods.

3.3 Manual testing

The five test cases were written with steps, input data and expected results in 02_TestCases.xlsx and 02_ManualResults.csv (columns test_id, test_name, input, category, assertion_type, expected_result, expected_min_count, steps, …). The manual run itself **has not been executed**: the log manual_execution_log.txt records that the environment used for that task had no browser, that only an HTTP HEAD check returned 200 (server reachable, not a functional result), and that no stopwatch time or screenshot was produced. The status of all five cases is therefore NOT RUN and the actual-result and duration columns are intentionally empty. As a consequence M is unknown.

3.4 Capture/replay suite

The flow was recorded with Playwright Codegen (a Selenium IDE project, search_products.side, is also provided) and then refined into search_products.spec.js: five tests, one per test case, each with its own hard-coded keyword and explicit assertions (visible heading, text of the “no results” element, product count). Explicit waits were added because the page loads data asynchronously. The suite is run with “npx playwright test” and timed three times with runner_benchmark.js. Table 5 shows where the 35 minutes of build time (S = 2,100 s) went.

Activity	Time (min)
Open recorder (Codegen / Selenium IDE)	3
Record the five test cases	12
Refine assertions (title text, no-results element, card count)	15
First replay and runner configuration	5
Total S (capture/replay)	**35**

Table 5. Build time of the capture/replay suite (recorded by the member).

3.5 Data-driven suite

One parameterised test function reads every row of test_data.csv through csv.DictReader and @pytest.mark.parametrize, so five rows produce five test runs and more cases can be added without touching the code. The column assertion_type selects the verification (Table 6). All selectors live in locators.py (Page Object pattern, class SearchLocators). After each case the script stores a screenshot and appends the result to a CSV log.

assertion_type	Check performed
exact_title	Displayed product title equals expected_result exactly (TC01)
contains_title	A displayed title contains expected_result, case-insensitive (TC02, TC03)
no_results	Element data-testid="no-results" shows “No products found.” (TC04)
min_count	Number of products in data-testid="product-list" ≥ expected_min_count = 6 (TC05)

Table 6. Assertion types driven by the CSV file.

Building this architecture took 50 minutes (S = 3,000 s): CSV schema and reader 10 min; SearchPage class and centralised locators 12 min; parameterised test and result resolver 18 min; pytest fixture, timing, screenshots and CSV logging 10 min. This is 15 minutes more than capture/replay, the price of a framework that separates data from logic.

3.6 Locator-change experiment

To measure maintenance, both suites were run with three outdated selectors that simulate a UI change (the experiment_broken_locators scripts), then repaired and re-run (experiment_fixed_locators scripts). Note that the application itself was not changed; the change is simulated by pointing the scripts to old selectors.

Element	Outdated locator	Correct locator
Search input	input.legacy-search-field	[data-testid="search-input"]
Category select	select#old-filter-category	[data-testid="category-select"]
Search button	button.btn-submit-search-v1	[data-testid="btn-search"]

Table 7. The three locators that were changed.

With the outdated locators both suites failed at all three interactions with “TimeoutError … Timeout 3000ms exceeded” (fill, select_option/selectOption, click). Repair effort differs: in capture/replay each test contains its own selectors, so every test must be edited; in the data-driven suite only the three lines of STANDARD_LOCATORS in locators.py change, and neither test_data.csv nor the test script is touched. After the repair the capture/replay suite passed again and the data-driven suite passed 5/5 rows.

Repair step (min)	Capture/replay	Data-driven
Read the failure log / locate the file	4	1.5
Inspect the DOM and edit the selectors	6	2.5
Re-run to confirm all tests pass	2	1.0
Total R_locator	**12 (720 s)**	**5 (300 s)**

Table 8. Time to repair three changed locators (recorded by the members). Data-driven is 2.4 times faster (58.3 % less).

3.7 Timing of the automated suites

Suite (5 test cases)	Run 1	Run 2	Run 3	Mean A	Pass rate
Capture/replay	11.51 s	11.07 s	11.15 s	11.24 s	100 % (15/15)
Data-driven	14.57 s	12.87 s	12.87 s	13.44 s	100 % (15/15)

Table 9. Machine-measured run times (headless Chromium, 3 consecutive runs). Per-test-case values are in Appendix B.

A standalone “python -m pytest” run of the data-driven suite took 27.32 s of test time for the five cases (36.73 s for the session) because each case uses an isolated browser context and takes two full-page screenshots; the benchmark figure of 13.44 s uses the same conditions as the capture/replay benchmark and is the one used in the model.

Table 9b gives the mean time of each test case over the three runs. All cases of both suites passed in every run, so the pass rate is 100 % (15/15 executions per suite). The data-driven cases are about 0.4–0.5 s slower per case, mostly because each case is executed in a fresh browser context created by the pytest fixture.

Test case	Capture/replay (s)	Data-driven (s)	Difference (s)
TC01 Exact Name Search	2.35	2.87	+0.52
TC02 Partial Name Search	2.19	2.59	+0.40
TC03 Keyword + Category	2.20	2.60	+0.40
TC04 Non-Existent Product	2.17	2.61	+0.44
TC05 Empty Keyword	2.32	2.77	+0.45

Table 9b. Mean execution time per test case (3 runs).

4. Results and discussion

4.1 Summary of measurements

Criterion	Manual	Capture/replay	Data-driven
Build time S	—	35 min (2,100 s)	50 min (3,000 s)
Run time per run A (5 TC)	not measured	11.24 s	13.44 s

Criterion	Manual	Capture/replay	Data-driven
Maintenance R per run (K = 30)	—	24 s	10 s
Pass rate (3 runs × 5 TC)	not run	100 % (15/15)	100 % (15/15)
Time to fix 3 locators	—	12 min (720 s)	5 min (300 s)
Where locators are edited	—	in each test (5 tests)	one file, 3 lines

Table 10. Comparison of the three methods (A: machine-measured; S and R_locator: recorded; R: converted with the assumption K = 30).

4.2 Break-even analysis

Answer to the main question – what the data support. The break-even point against manual testing cannot be computed yet, because M was not measured. What the measurements do establish is the condition under which automation can pay off and the formula that gives N_BE once M is known (Table 11). For example, with K = 30 a manual run of the five cases must take longer than about 35 s for capture/replay, and longer than about 23 s for data-driven, otherwise no break-even exists.

Option	A + R (s)	M must exceed	N_BE (s)
Capture/replay, UI stable (R = 0)	11.24	11.24 s	2100 / (M − 11.24)
Capture/replay, K = 30 (R = 24)	35.24	35.24 s	2100 / (M − 35.24)
Data-driven, UI stable (R = 0)	13.44	13.44 s	3000 / (M − 13.44)
Data-driven, K = 30 (R = 10)	23.44	23.44 s	3000 / (M − 23.44)

Table 11. Break-even conditions and formulas expressed in the unknown M.

Comparison between the two automated suites (independent of M). With K = 30 the per-run cost is 35.2 s for capture/replay and 23.4 s for data-driven, so the extra 900 s of build effort of the data-driven suite is recovered after about 76 runs: from run 77 onward data-driven has the lower cumulative cost (Figure 2). If the UI never changed (R = 0) capture/replay would remain cheaper at every N, because it has both lower S and lower A.

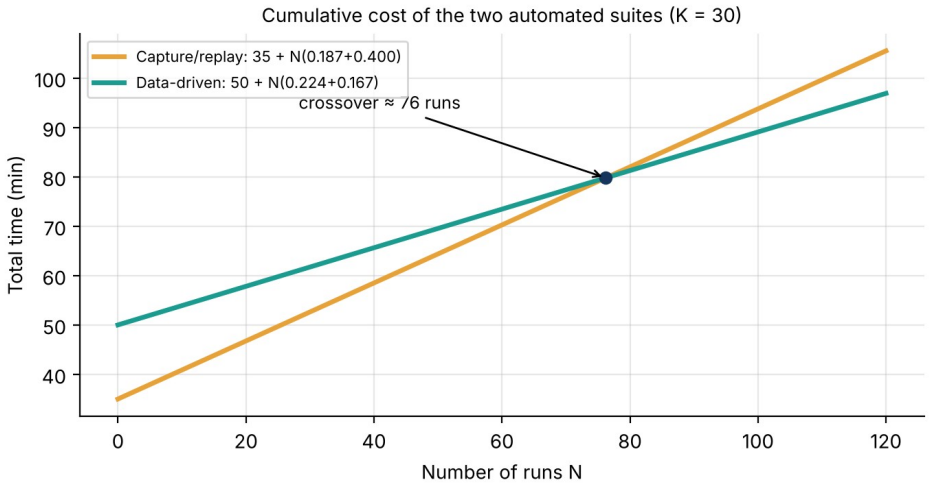


Figure 2. Cumulative cost of the two automated suites; the manual line N × M can be added once M is measured (05_BreakEven.xlsx).

What-if illustration (assumed M, not a result). The workbook sheet "Sensitivity (assumed M)" shows that the break-even depends strongly on M: if one manual run of the five cases took 3 minutes, automation would be cheaper after 13–15 runs (capture/replay) or 19–20 runs (data-driven); at 2 minutes the figures become 20–25 and 29–32. These values are scenarios only and must be replaced by the measured M.

4.3 Link to the seven testing principles

- **Principle 1 – Testing shows the presence of defects, not their absence.** Both suites passed 15/15 executions, which only means that no regression was found within these five search scenarios; it does not prove that the search feature or the site is defect-free.
- **Principle 5 – Pesticide paradox.** Repeating the same tests with the same data eventually finds no new defects. Capture/replay hard-codes its data; the data-driven suite lets testers add boundary or special-character rows to test_data.csv without code changes, which makes it easier to refresh the tests.
- **Principle 6 – Testing is context dependent.** No approach is best everywhere: with a short project and a stable UI, the lower S of capture/replay wins; with a long-lived product whose locators change, the low maintenance of the data-driven suite pays back after about 76 runs under our assumption.

4.4 Limitations

- Only three runs were timed per suite and only one flow with five test cases was tested.
- The manual run is missing, so M and the break-even point against manual testing are open.
- The two automated suites were timed on different dates (2026-11-03 and 2026-11-07), and results depend on network latency to the hosted application.
- S and R_locator are durations recorded by the members, not stopwatch-controlled measurements; the locator change was simulated with old selectors.
- R depends on the assumed cycle K = 30; a different K changes the amortised maintenance and the crossover point.

4.5 Practical recommendations

- **Measure M first.** One timed manual run of TC01–TC05 (with screenshots) completes the model; until then the team can only quote the conditions and formulas of Table 11.
- **Choose by horizon.** For a short project or a demonstration with a stable UI, capture/replay is the cheaper start (35 min). For a suite that will run many dozens of times or whose interface keeps changing, the data-driven suite with centralised locators is the better investment.
- **Reduce R at the source.** Both suites already use stable data-testid attributes; agreeing these identifiers with the developers is the cheapest way to avoid locator repairs altogether.
- **Do not replace thinking with tooling.** Automated regression checks guard against regressions only; exploratory manual testing is still needed to find new defects (Principle 5 and the risk of over-reliance in SWT6).
- **Report assumptions with the numbers.** Every figure that depends on K, on recorded durations or on a simulated change should be quoted together with that assumption.

5. Conclusion and references

Within the product-search flow and measurement conditions of this project: (1) both automated suites ran reliably, passing 15/15 executions in 11.24 s (capture/replay) and 13.44 s (data-driven) for five test cases; (2) the data-driven suite costs 15 minutes more to build (50 vs 35 min) but repairs three changed locators in 5 minutes instead of 12, and under K = 30 it becomes cheaper than capture/replay after about 76 runs; (3) the number of runs after which automation beats manual execution cannot yet be stated because M was not measured – the model $N_{BE} = S / (M - A - R)$ is ready, and the next step is to time one manual run and enter it in 05_BreakEven.xlsx. Automation is worthwhile when runs are repeated often and its maintenance cost is kept under control.

References

[1] R. Black, E. van Veenendaal, D. Graham, Foundations of Software Testing – ISTQB Certification, 3rd ed., Cengage Learning, 2012, chapter 6, pp. 169–196.

[2] Software Testing course, slides SWT6 “Tool Support for Testing”, slides 37–39 (benefits and risks of tools).

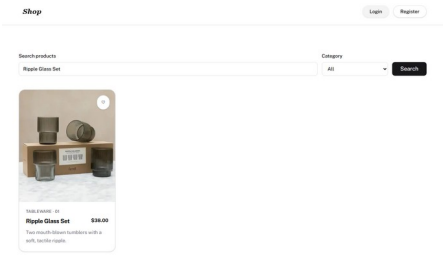
[3] ISTQB, Certified Tester Foundation Level Syllabus (seven testing principles).

[4] Playwright documentation, https://playwright.dev/

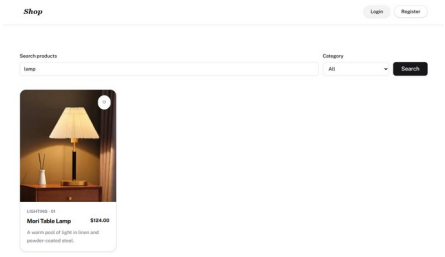
[5] pytest documentation, https://docs.pytest.org/

Appendix A. Test cases and evidence

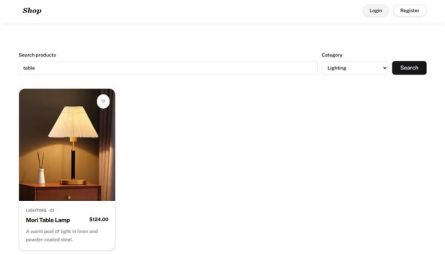
Detailed test-case steps (from 02_TestCases.xlsx): open the home page, type the keyword (and choose the category for TC03), press Search, compare the displayed result with the expected result in Table 4. Screenshots below were produced by the data-driven suite (04_DataDriven/evidence).



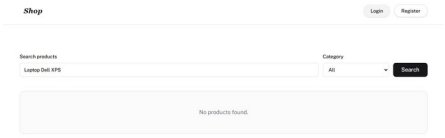
TC01 – exact name search



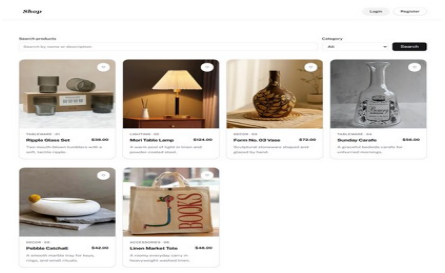
TC02 – partial name search



TC03 – keyword + category



TC04 – no results



TC05 – empty keyword (6 products)

Appendix B. Timing logs

Per-test-case times in seconds (source: 03_CaptureReplay_Metrics.csv and 04_DataDriven_Metrics.csv). Every execution has status PASS.

Test case	CR 1	CR 2	CR 3	CR mean	DD 1	DD 2	DD 3	DD mean
TC01	2.60	2.20	2.25	2.35	3.56	2.45	2.59	2.87
TC02	2.19	2.19	2.20	2.19	2.66	2.65	2.46	2.59
TC03	2.20	2.20	2.21	2.20	2.82	2.56	2.42	2.60
TC04	2.19	2.17	2.14	2.17	2.70	2.49	2.63	2.61
TC05	2.33	2.31	2.34	2.32	2.83	2.70	2.77	2.77
Suite	11.51	11.07	11.15	11.24	14.57	12.87	12.87	13.44

Table B1. CR = capture/replay, DD = data-driven.

Appendix C. How to reproduce

Capture/replay (folder 03_CaptureReplay): “npm install”, then “npx playwright test” for the suite and “node runner_benchmark.js” for the three-run benchmark; experiment_broken_locators.js and experiment_fixed_locators.js run the locator experiment.

Data-driven (folder 04_DataDriven): “pip install -r requirements.txt”, “python -m playwright install chromium”, then “python -m pytest” for the suite; runner_benchmark.py produces the timing CSV/JSON; experiment_broken_locators.py and experiment_fixed_locators.py run the locator experiment.

Break-even workbook: open 05_BreakEven.xlsx (Group_Topic5_BreakEven.xlsx), enter the measured manual time M (minutes) in cell C5 of sheet “Break-even Model”.